

Profiling & Performance

Week 9 Workshop

Yongqiang Tian

Department of Software Systems and Cybersecurity
Faculty of IT, Monash University

Acknowledgement

I wish to acknowledge the people of the Kulin Nations, on whose land we are gathered today. I pay my respects to their Elders, past and present.

Why do we need profiling?

Suppose your teammate sends you `report.py`.
It reads transaction records and produces a report
for selected customers.
The report contains the expected results.

Current run

30 seconds

Report must be saved within

5 seconds

The total runtime tells us there is a problem. Profiling helps us locate where the time goes.

Profiling and benchmarking

Profiling measures how different parts of a running program use time and other resources.

Measurement	What it tells us
Benchmark	Performance for a particular task and input.
Profile	Which parts of the program account for the cost.

Use a profile to choose where to investigate or change the code.

Use a benchmark to check whether the change improved performance.

Where bottlenecks can occur

A **bottleneck** limits how quickly a task can finish.

Possible source of cost	Example operation
Repeated computation	Search a long customer list for every record.
Waiting for I/O	Wait for a file read or a server response.
Allocation and copying	Create and copy large temporary arrays.
Device transfers	Move data between CPU and GPU memory.

A short operation repeated many times can cost more than a long function called once.

What should we measure?

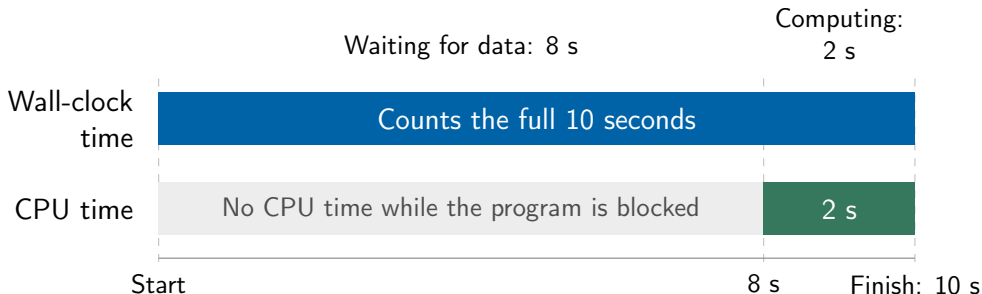
Choose measurements that answer the performance question.

Measurement	Question it helps answer
Wall-clock time	How long does the task take, including waiting?
CPU time / activity	How much CPU work occurs? How busy is the CPU?
Memory	What is the peak usage? Where is memory allocated?
GPU time / memory	What time and memory do device operations and transfers use?

These measurements can describe the same run. Use them together to investigate its behaviour.

Wall-clock time and CPU time

Suppose a program waits 8 seconds for data, then computes for 2 seconds.



Wall-clock time is also called **elapsed time**: time from start to finish.

CPU time counts time the CPU spends executing this program.

What memory measurements mean

Current usage

Memory in use at one moment.

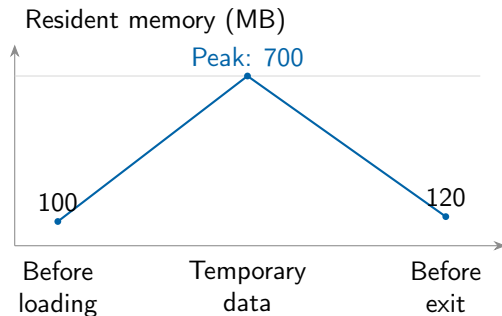
Peak usage

Highest observed usage during the run.

Allocation sites

Code that requests memory to create objects.

Suppose memory use changes like this:



A process monitor measures resident RAM. `tracemalloc` locates Python allocations.

What to profile on a GPU



Execution time	Time spent on device operations.
Transfer time	Time spent moving data to and from the GPU.
Activity over time	Intervals when the GPU is working or idle.
Device memory	Memory used on the GPU.

Transfers can dominate the time needed to return a result, even when GPU computation is fast.

How do we profile a program?

- 1 Define the input, required output and start/stop points.
- 2 Measure the original performance repeatedly, with the profiler off. This is the **baseline**.
- 3 Collect a profile and locate expensive functions, lines or operations.
- 4 Inspect the code, make a relevant change and check the output.
- 5 Repeat the original performance measurement to see whether the target is met.

Choosing a tool for the question

Question	Example tool
How long does the task take?	<code>time</code> / an elapsed timer
Which Python functions take time?	<code>cProfile</code>
Which lines in that function take time?	<code>line_profiler</code>
How much memory? Which allocations?	Process monitor / <code>tracemalloc</code>
Which GPU operations or transfers take time?	PyTorch Profiler

For a Python script named `report.py`:

```
python3 -m cProfile -s cumulative report.py
```

`-s cumulative` sorts by time including called functions.

Reading a function profile

Suppose a 27-second function call spends its time in two other functions.

Function	ncalls	tottime	cumtime
process_records	1	0 s	27 s
find_matches	1	24 s	24 s
calculate_totals	1,000	3 s	3 s

`ncalls` Number of calls.

`tottime` Self time across all calls, excluding called functions.

`cumtime` Time across all calls, including called functions.

`process_records` includes its called functions: $27 = 24 + 3$ seconds.

The 3 seconds for `calculate_totals` cover all 1,000 calls.

From a function to an expensive operation

```
def find_matches(records, wanted_ids):  
    matches = []  
    for record in records:  
        if record.customer_id in wanted_ids:  
            matches.append(record)  
    return matches
```

wanted_ids is the list of customers requested for the report.

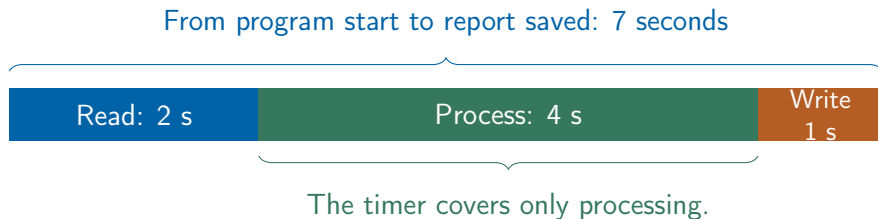
With [7], keep a record for customer 7 and skip one for customer 2.

Each record starts another list search. If the ID is absent, the search reaches the end.

A function profile locates this function. A line profile can test whether repeated list searches account for its time.

Pitfall: timing only part of the task

Suppose a timer measures only the 4-second processing stage.



The timing must cover the task the user needs completed.

Here, the user waits **7 seconds** from program start to report saved.

Pitfall: comparing different conditions

What can go wrong	How to make the comparison meaningful
Different input or output	Use the same required task. Check the output after each change.
One unusually fast run	Repeat measurements under relevant machine, cache and startup conditions.
Profiler adds overhead	Turn it off when comparing final performance against the baseline.

Keep the individual runtimes. A single favourable number can hide variation.

Pitfall: drawing the wrong conclusion

Observation	What we still need to investigate
The CPU is busy.	Which function or operation accounts for the work?
Memory reaches a high peak.	Is temporary data released, or does unneeded data remain across tasks?
A Python call returns quickly.	Has the submitted GPU work finished and the required result returned?

Check what the tool measured before drawing a conclusion.

- Python: clocks and profiling
- Python: allocation tracing
- line_profiler documentation
- PyTorch: profiling and GPU timing